The 8-point grid and why every good app uses one

Week 5, Lecture 1, Design for Builders: Ship Beautiful Products as a Founder

Autumn 2026

What we will cover

- Why 8px is the right base unit for most UI work
- When to use 4px half-steps and when not to
- Spacing tokens: naming your values so they survive a redesign
- T-shirt sizing: xs, sm, md, lg, xl as a shared design vocabulary
- Inner padding vs outer margin: the distinction that makes or breaks a component

Part 1: the case for 8

The pixel density problem

Modern screens do not all use the same pixel density.

- iPhone at 2x retina: 1 logical pixel = 2 physical pixels
- High-DPI at 3x: 1 logical pixel = 3 physical pixels
- A value like 9px does not divide evenly into 2 or 3
- The device must render it as a fractional pixel: slightly blurry, slightly off

Multiples of 8 divide evenly into 2, 4, and 8. They render cleanly at every common density.

8px is also the right perceptual step

Beyond device math, 8px is the right step size for human perception.

- The difference between 8px and 16px is visible to the eye
- The difference between 16px and 17px is not
- A finer grid gives you more choices and more inconsistency
- A coarser grid gives you fewer choices and more visual coherence

The Spec Network (2016): "When you use multiples of 8 to size and space, you are working within a system that maps cleanly to all device resolutions currently on the market."

Not rules: constraints

The 8-point grid is not a rule you follow for its own sake.

- It is a constraint that removes low-value decisions
- Without a grid, every spacing decision is made from scratch
- With a grid, the question "is 14px or 16px right here?" does not exist
- You choose from a small set of valid values, and the choice is meaningful

Fewer options. Faster decisions. More consistent output.

Part 2: the 4px half-step

When 8px is too coarse

The 8-point grid is the right scale for spacing between components.

Inside a component, 8px can be too large.

- A button with 8px top padding and 16px side padding is fine
- A chip or badge with 8px padding on all sides looks swollen
- An icon and its label with 8px gap look like two separate things

The 4-point half-step fills the gap.

The rule for choosing between 8 and 4

Use the 8-point grid for **outer spacing** (between components).

Use 4-point half-steps for **inner spacing** (within a component).

- Outer: gap between two cards, margin from card to column edge, section-to-section vertical rhythm
- Inner: padding inside a button, gap between icon and label, space between a form field and its error text

The 4-point values

8-point grid values: 8, 16, 24, 32, 40, 48, 64

4-point half-steps: 4, 12, 20, 28, 36

All 4-point values are valid 8-point values: there are no values that are on the 4-point grid but off the 8-point grid. The 4-point grid is a refinement, not a replacement.

Part 3: spacing tokens

The problem with raw pixel values

If you type "16" into a Figma padding field, you have a pixel value, not a decision.

- What does 16 mean in this design system?
- Is it the standard inner padding, or a special-case outer gap?
- When you want to change all "standard inner padding" values, how do you find them?

The answer to all three questions is a token.

What a spacing token is

A spacing token is a named reference to a specific pixel value.

- Token name: `space-md`
- Token value: 16px

You use the token name everywhere. The pixel value lives in one place.

- Change `space-md` from 16 to 20, and every component using `space-md` updates
- Read a component spec and see `space-md`, and you know exactly what spacing relationship is intended

Tailwind Labs (2024): every `px-*` and `gap-*` utility is a spacing token. `p-4` = 16px. `p-8` = 32px. The class name is the token.

T-shirt sizing

T-shirt sizing is a naming convention for spacing tokens.

```
xs  =  4px
sm  =  8px
md  = 16px
lg  = 24px
xl  = 32px
2xl = 48px
3xl = 64px
```

The exact values depend on the product. A dense data tool might set `md` to 12px. A consumer app might set it to 20px. The names stay the same. The scale stays proportional.

T-shirt sizing in practice

Which token for which situation:

- `xs` (4px): icon-to-label gap, small badge padding, help text offset
- `sm` (8px): tight button padding, dense list-item gap, inline tag padding
- `md` (16px): standard card padding, standard form field padding, default gap
- `lg` (24px): comfortable outer gap between cards, section sub-group spacing
- `xl` (32px): major layout gap, spacious hero padding
- `2xl` (48px): section-to-section separation
- `3xl` (64px): page-level vertical rhythm

Part 4: inner padding vs outer margin

Two kinds of space

Every spacing relationship in a UI is one of two things.

Inner padding: space between a component's edge and its content.

- Changes the component's size
- Controls how dense or airy the component looks

Outer margin (gap): space between a component and its neighbors.

- Changes the component's position relative to other components
- Controls how related or separate components look from each other

The proximity rule

Outer spacing must be larger than inner spacing at the same level of the hierarchy.

If a card has 16px inner padding, the gap between cards should be 24px or more.

If the gap between cards is 12px and the inner padding is 16px, the space between cards reads as tighter than the space inside them. The eye reads the gap as interior space, not separation.

This is the Gestalt proximity principle from Week 2 expressed as a spacing number.

The anti-pattern: equal spacing everywhere

The most common beginner spacing mistake: using the same value for inner padding and outer gaps.

Every element at 16px padding, 16px gap.

The eye cannot distinguish between "inside this component" and "between components." The layout looks like one undifferentiated mass of content.

Fix: inner spacing uses one token, outer spacing uses the next token up.

A card example

```
Card component
  padding: md (16px)            ← inner

  Icon to headline gap: sm (8px) ← inner, tighter subgroup
  Icon to label gap: xs (4px)    ← inner, single semantic unit

Cards in a row
  gap: lg (24px)                ← outer, larger than inner
```

The 24px gap signals: these are separate things.
The 16px padding signals: this content lives together.
The 8px gap signals: these elements form a cluster.
The 4px gap signals: this is one compound element.

Take-aways

1. Multiples of 8 render cleanly at every common device pixel density. Use them for all outer spacing between components.
2. 4px half-steps are for inner spacing within dense components. They are a refinement of the 8-point grid, not a different system.
3. Spacing tokens with T-shirt names make decisions explicit, searchable, and refactorable. Every spacing value in your product is either a token or a bug.
4. Inner padding controls component density. Outer gap controls component separation. Outer spacing must be larger than inner spacing at the same hierarchy level.
5. The 8-point grid removes a class of low-value decisions. The spacing scale that remains tells the eye what belongs together and what is separate.

> "When you use multiples of 8 to size and space, you are working within a system that maps cleanly to all device resolutions currently on the market."
>
> Spec Network, "The 8-Point Grid," spec.fm, 2016

What's next

- **This week, Lecture 2:** Grids, columns, and responsive layout in Figma
- **This week, section:** Pixel-grid drill: rebuild three real product screens on an 8-point grid
- **Reading:** Week 5 reading at `/c/design-for-builders-26au/readings/wk05`
- **HW2 due this week:** Type-only redesign. Bring to section before submitting.